CISGUARD 360 / TEAM ENGINEERING

# Vidhi - Parallel Development Playbook

DEVICE ENGINE + SECURITY AUTOMATION

Phase-by-phase AI IDE prompts, libraries, deliverables, tests, and independent merge contract. Original problem statement remains mandatory; no features removed.

| PRODUCT SCOPE | NON-NEGOTIABLE | WORK MODE |
| --- | --- | --- |
| 88 features: M01-M38 + E01-E30 + I01-I20 | No runtime LLM/AI/ML, no fake results, read-only auditing | Parallel work first; contract-based merge later |

## Your mission

All specified OS/firewall/router/switch collectors, OpenSCAP/ComplianceAsCode, parsing, what-if models, hardening, rollback, verification.

## Explicit ownership boundary

Cannot own Supabase schemas/Auth or frontend pages; must emit v1 contracts and run without main API.

## Required reference uploads

• Inputs to upload in Phase 0: CISGuard_360_Complete_PRD_v1.pdf and CISGuard_360_Final_Tech_Stack_v3.pdf.

• Scope is exactly M01-M38 (38), E01-E30 (30), I01-I20 (20) as documented in PRD; feature IDs never change.

This guide is a development work plan, not an assertion that the system or any vendor integration is already complete.

# 1. Team split and parallel-work rules

| Member | Exclusive working directories | Main responsibilities |
|---|---|---|
| Apoorva | apps/web/; docs/frontend/ | All pages, UX, graph/simulation visuals, frontend testing |
| Mandar | apps/api/; packages/contracts/; infra/; docs/core/ | Backend lead, Supabase DB/Auth/RLS, CIS rules, risk graph, optimizer, reports |
| Vidhi | services/device-engine/; docs/device/ | All platform adapters, SCAP, policy collectors, simulation, hardening, verification |

```
SINGLE-OWNER REPOSITORY LAYOUT
apps/web/                 -> Apoorva only (frontend; UI mocks and UI tests)
apps/api/                 -> Mandar only (FastAPI, Supabase, policy, risk, optimizer,
reports)
services/device-engine/   -> Vidhi only (platform adapters, SCAP, simulation, hardening,
 verification)
packages/contracts/v1/    -> Mandar owns; changes require all three to agree
infra/                   -> Mandar owns (Docker, env examples, queue, Supabase setup)
docs/frontend/            -> Apoorva; docs/core/ -> Mandar; docs/device/ -> Vidhi

PARALLEL WORKING RULES
1. Work in your own folder/branch: feat/apoorva-web, feat/mandar-core,
feat/vidhi-device.
2. Do NOT edit teammates' owned directories; mock missing dependencies.
3. All production payloads use contracts/v1; do not introduce parallel schemas.
4. Start Phase 0 by reading the two uploaded source PDFs completely.
5. Tag every ticket/test by Mxx, Exx, Ixx and record DONE / PARTIAL / BLOCKED /
UNTESTED.
6. Read-only scan is the default; live device writes only after approval and safeguards.
7. No AI/ML/LLM inside the product, and no paid third-party auditing API dependency.
8. Never fabricate compliance scores, findings, device compatibility, or successful
remediations.
9. All samples and screenshots use synthetic data; no device credentials in repo/AI
prompts.
10. A phase finishes only after code, tests, docs, and handoff artifacts are produced.
```

## How three people work without waiting for each other

• Apoorva: use MSW or a typed in-memory adapter to render every screen and user journey while API is unavailable.

• Mandar: use MockDeviceEngine implementing contract v1 until Vidhi package is installable.

• Vidhi: run a standalone CLI/fixtures and contract tests without importing Mandar API or Supabase.

• Contract changes: propose schema diff, review with all three, publish a minor version and only then update consumers.

• Track M/E/I feature IDs in every phase, issue, pull request and acceptance test; full integration requires evidence.

# 2. Technology, libraries and IDE skills

| Area | Exact tools / libraries |
|---|---|
| OS audit | PowerShell, WinRM/pypsrp/pywinrm, auditpol, secedit, Microsoft SCT/LGPO, SSH, OpenSCAP, ComplianceAsCode, osquery |
| Network audit | Netmiko, Scrapli, Paramiko, Nornir, TextFSM, TTP, ciscoconfparse2, pyATS/Genie, HTTPX, dnspython, pysnmp |
| Network modeling | Batfish + pybatfish, custom vendor semantics, NetworkX for local dependency graphs, fixture topology corpus |
| Hardening | Ansible Core, Ansible Runner, ansible.windows and approved vendor collections, PowerShell, vendor APIs, OpenBao worker client |
| Validation lab | pytest, Hypothesis, Ruff, mypy, GNS3, EVE-NG, Windows/Linux VMs, Wireshark, tcp/SSH/HTTP probes |
| Development skills | vendor-adapter-builder, openscap-compliance-content, safe-remediation, network-model-validation, no-fake-results, fixture-driven-vendor-testing |

### Skill-file rule

The named development skills above are proposed reusable instruction files, not installed IDE capabilities. If the AI IDE supports project skills, create them under a local skill/instructions directory. Otherwise keep equivalent markdown checklists under your docs folder. Skills guide code generation; they are NOT runtime AI modules.

Cross-team core stack remains Python/FastAPI, Supabase DB/Auth, OpenSCAP + ComplianceAsCode for compatible Linux checks, NetworkX, OR-Tools, Batfish/custom simulation and Ansible. Use only your owned layers; do not reimplement teammates' tools.

### Shared developer tooling

Git + branches, GitHub pull requests, Docker/Compose, dependency lockfiles, .env.example, structured logs, CI test commands, signed-off handoff notes. For any vendor hardware, use authorized device access or licensed lab images only.

# 3. Member-specific implementation inventory

Every original PS device family remains included. Adapter matrices must distinguish fixture-tested, lab-tested, unsupported firmware and manual-only checks - do not claim universal support based on one configuration sample.

| Required target | Collection / security tooling | Facts and validation focus |
|---|---|---|
| Windows 11 Enterprise/Standalone | PowerShell, WinRM, auditpol, secedit, registry, osquery | Authentication, firewall, services, audit logs, OS controls |
| Windows Server 2019 | PowerShell, WinRM, Microsoft SCT/LGPO | Domain/local policies, effective audit settings, OS controls |
| Ubuntu 20.04 LTS | SSH, native tools, custom rules; compatible SCAP when possible | PAM, sshd, auditd, UFW, permissions, sysctl |
| Ubuntu 22.04 LTS | OpenSCAP/ComplianceAsCode where mapped, SSH | PAM, sshd, auditd, nftables/UFW, permissions |
| RHEL 8/9 | OpenSCAP, SCAP Security Guide, SSH | SELinux, firewalld, auditd, PAM, SSH, services |

| Required target | Collection / security tooling | Facts and validation focus |
|---|---|---|
| Fortinet FortiOS | REST API, SSH, parser | Ordered policy, management security, syslog/NTP/SNMP |
| Cisco ASA / FTD | ASA CLI or FMC API, version-specific adapters | ACL/NAT, management plane, security policy |
| Check Point | Management API + Gaia access as authorized | Access rule order, objects, routing and management |
| Sophos Firewall | Documented firmware-specific XML/REST API | Firewall rules, NAT, objects, management settings |
| Palo Alto PAN-OS | REST/XML API + commit model | Security policy, zones, services, management |
| Cisco router | Netmiko/Scrapli/pyATS + parser | ACL, routing, SSH, NTP, syslog, SNMP |
| Cisco managed switch | Cisco CLI/pyATS + parser | Port security, unused ports, DAI, DHCP snooping, VLAN 1 |

### Network control families requiring actual evidence

| Feature IDs | Checks | Correctness constraints |
|---|---|---|
| M20-M21 | Telnet/HTTP management, SSHv2, key auth, session timeout | Per-interface/plane evidence and device family applicability |
| M22-M25 | Broad ACLs, sensitive ingress ports, shadowed ordered rules, default-deny | Correct rule ordering, NAT/zone/context/implicit action |
| M26-M30 | Unused ports, port security, DHCP snooping, DAI, management VLAN 1 | Port/VLAN/edge configuration and designated-unused inventory |
| M31-M33 | Remote syslog, NTP synchronization, SNMPv3 and insecure v1/v2c | Configuration + sync/telemetry state where available |

### Adapter contract, safe execution and lab readiness

- Use one canonical typed envelope for collection and evidence, independent from Supabase.
- Use OpenSCAP/ComplianceAsCode where compatible, but verify CIS mapping and Ubuntu 20.04 gaps.
- Maintain vendor/model/firmware capability and tested-version matrices for collection/simulation/remediation.
- Separate read-only collect from approved execute; never run a raw browser-provided command.
- Verify successful post-change CIS controls and necessary service connectivity independently.
- Add secure/weak/unknown fixture cases and real authorized lab evidence when available.

## 4. Frozen v1 contract - identical for all three

Read this carefully before giving any phase prompt to an AI coding IDE. It prevents three independently generated projects from becoming incompatible.

```
CONTRACT v1.0.0 - FREEZE BEFORE INTEGRATION
Base HTTP prefix: /api/v1. Supabase Auth provides Bearer access token.
All API routes validate tenant membership/permissions on the backend.
Identifiers: org_id, asset_id, scan_id, finding_id, policy_pack_id,
plan_id, simulation_id, change_id, verification_id, report_id (UUID strings).
UTC timestamps: ISO 8601. Pagination: cursor + limit; filters in query params.
Status enum: PASS | FAIL | NOT_APPLICABLE | MANUAL_REVIEW | ERROR |
NOT_ASSESSED. EXCEPTION is a separate approval overlay, not fabricated PASS.
Simulation verdict: SAFE_IN_MODEL | VIOLATES_CONTRACT | UNSUPPORTED |
INSUFFICIENT_EVIDENCE. Execution states: DRAFT, SIMULATED, PRECHECKED,
PENDING_APPROVAL, APPROVED, EXECUTING, VERIFYING, VERIFIED, FAILED,
ROLLED_BACK, NEEDS_MANUAL_RECOVERY.
Canonical API error: {code, message, request_id, details?, retryable}.
Canonical resource envelope: {schema_version:'1.0.0', data, meta:{request_id}}.
Audit scores are server-computed: PASS/(PASS+FAIL); show assessed coverage.
Collector evidence must include asset_id, org_id, platform, vendor,
model, os_or_firmware_version, collector_version, collection_method,
collected_at, facts, evidence_refs[], supported_capabilities[],
unsupported_capabilities[], errors[]. Each evidence_ref has sha256 hash,
source command/API identity, encrypted artifact location and timestamp.
No raw passwords, keys, secrets, or privileged token contents in contracts.
Worker communication: JSON-serializable Pydantic commands/results; versioned.
Do not store free-form executable commands received from the browser.
```

## Minimum canonical examples

```
GET /api/v1/scans/{scan_id}
{ "schema_version":"1.0.0", "data": {
    "scan_id":"uuid", "org_id":"uuid", "status":"RUNNING",
    "requested_assets":3, "completed_assets":1,
    "assessed_compliance_pct":75.0, "assessment_coverage_pct":60.0
  }, "meta":{"request_id":"uuid"}}

Normalized evidence:
{ "schema_version":"1.0.0", "org_id":"uuid", "asset_id":"uuid",
  "platform":"cisco_ios", "vendor":"Cisco", "model":"lab-fixture",
  "os_or_firmware_version":"example", "collector_version":"0.1.0",
  "collection_method":"offline_fixture", "collected_at":"UTC timestamp",
  "facts":{"management":{"telnet_enabled":true}},
  "evidence_refs":[], "supported_capabilities":["management_telnet"],
  "unsupported_capabilities":[], "errors":[] }
```

### API route families agreed for v1

| Domain | Endpoint families |
|---|---|
| Auth + organizations | /auth/session; /organizations; /organizations/{id}/members; /me/mfa |
| Assets + inventory | /assets; /assets/{id}; /assets/discover; /assets/{id}/capabilities |
| Policy library | /policies; /policies/{id}; /policies/{id}/rules; /policies/{id}/diff; /exceptions |
| Audit queue | /scans; /scans/{id}; /scans/{id}/assets; /scans/schedules |
| Findings + evidence | /findings; /findings/{id}; /evidence/{id}; /audit-history |
| Graph + risk | /graph; /risk-scenarios; /risk-scenarios/{id}; /trends |
| Plans + simulation | /plans; /plans/{id}; /simulations; /simulations/{id}; /safety-contracts |
| Approval + hardening | /approvals; /changes; /changes/{id}; /changes/{id}/events |
| Verification + proof | /verifications; /proof-packages; /proof-packages/{id} |
| Reports + integrations | /reports; /reports/{id}/download; /notifications; /integrations |

Any field and route detail not specified here must be resolved in contracts/v1 before implementation. The two source documents remain authoritative for functional behavior; this contract is authoritative for integration wiring.

# 5. Your phase sequence and copy-paste prompts

Workflow: Upload both source PDFs to your own AI IDE conversation, paste Phase 0 prompt, inspect the actual output and tests, then paste Phase 1, etc. Keep the same project workspace and phase handoff files. Prompts are independent between team members, but sequential within one member.

### PHASE 0 | Source audit, vendor coverage, device-engine interface

Read references; set adapter contract, platform matrix and fixture governance.

Tools/libraries this phase: PRD/stack, Pydantic contracts, capability matrix

Suggested AI IDE skills/checklists: device-contracts, coverage-tracking

Deliverables / exit gate: Device-engine API contract notes, all platforms map, fixtures list, no direct DB dependency.

**COPY-PASTE PROMPT | PHASE 0**

```
You are Vidhi, DEVICE INTEGRATION + COMPLIANCE COLLECTION + SIMULATION + HARDENING
backend owner for CISGuard 360. I upload CISGuard_360_Complete_PRD_v1.pdf and
CISGuard_360_Final_Tech_Stack_v3.pdf. Read BOTH fully, retain all 88 features, and
follow shared contract appendix. Work only services/device-engine and docs/device; you
must be able to run against local synthetic fixtures without Mandar's API or Apoorva's
UI. Build a Python package, public typed interfaces, JSON input/output CLI, explicit
capability matrices by vendor/model/firmware, evidence provenance and read-only
default. No runtime AI/ML, no paid auditing API, no plaintext secrets. Produce
docs/device/phase0-coverage.md, adapter matrix for M01-M15/M20-M33, test fixtures plan,
 collection security checklist and versioned interface stubs.
```

### PHASE 1 | Device-engine skeleton, fixtures and canonical normalization

Implement universal SDK, CLI and evidence envelope before vendors.

Tools/libraries this phase: Python, Pydantic, Netmiko, TTP, TextFSM, pytest

Suggested AI IDE skills/checklists: collector-sdk, golden-fixture-tests

Deliverables / exit gate: Installable package, typed adapter registry, deterministic fixtures and tests.

**COPY-PASTE PROMPT | PHASE 1**

```
Initialize services/device-engine with pyproject.toml, Pydantic models,
Netmiko/Scrapli/Paramiko, HTTPX, TTP/TextFSM, lxml/PyYAML, proper logging and pytest.
Implement Adapter Protocol: probe_version(), collect(read_only=True), parse(),
normalize(), supported_controls(), remediation_capabilities(), optional
execute_approved_change(). Build canonical FactsEnvelope v1.0.0 and JSON CLI: probe,
collect, evaluate_device, candidates, simulate, execute, verify; local fixture replay.
Every result must include vendor version, collection source, evidence hash and
unsupported/errors; never pass invalid/unknown as secure. Add safe synthetic config
fixture corpus with expected normalized values, golden JSON tests, no remote access by
default.
```

## PHASE 2 | Windows 11 and Server 2019 full collectors

PowerShell, WinRM, local security policies and CIS evidence.

Tools/libraries this phase: PowerShell, WinRM, pypsrp, Microsoft SCT/LGPO, osquery

Suggested AI IDE skills/checklists: windows-policy-extraction

Deliverables / exit gate: Windows adapter and version fixtures, read-only collection tests, evidence coverage.

**COPY-PASTE PROMPT | PHASE 2**

```
Implement Windows 11 Enterprise/Standalone and Windows Server 2019 audit collectors
using approved PowerShell/WinRM/pypsrp/pywinrm, registry, CIM/WMI, auditpol, secedit,
service/firewall state, local and effective policy. Include Microsoft's Security
Compliance Toolkit LGPO/Policy Analyzer as supporting diagnostic tooling; Microsoft
baselines are NOT CIS equivalents. Where evidence requires local agent or privileged
command, specify exact privileges and unknown outcomes. Build normalized auth, account
policies, audit log, firewall, services and permissions facts; write
secure/weak/unknown fixture tests per OS family. No write operation in auditing path.
Provide per-control evidence mapping to CIS policy engine maintained by Mandar. Cover
M01,M02,M14.
```

## PHASE 3 | Ubuntu and RHEL collectors + OpenSCAP/ComplianceAsCode

Linux native collectors; supplemental SCAP content; CIS mapping.

Tools/libraries this phase: SSH, OpenSCAP oscap, ComplianceAsCode, SCAP Guide, osquery

Suggested AI IDE skills/checklists: linux-CIS-mapping, SCAP-integration

Deliverables / exit gate: Ubuntu 20.04/22.04 and RHEL 8/9 collectors, OS-specific fixtures and SCAP result mapping.

**COPY-PASTE PROMPT | PHASE 3**

```
Implement version-aware Ubuntu 20.04/22.04 and RHEL 8/9 SSH collectors with native
checks: sshd, PAM, auditd, accounts, sysctl, host firewalls, services, package/update
state, filesystem permissions, SELinux on RHEL where relevant. Integrate OpenSCAP oscap
 CLI and compatible ComplianceAsCode/SCAP Security Guide content where available;
ingest XCCDF/OVAL results with version and source metadata. Do NOT claim SCAP content
equals a CIS Benchmark automatically; map and validate each relevant CIS control,
implement additional custom deterministic checks for missing content. Explicitly handle
 Ubuntu 20.04 if current ComplianceAsCode content does not include it. Provide
controlled Linux VM tests and synthetic fixtures for both RHEL major and Ubuntu LTS
versions. Cover M03,M04,M14 and relevant M20,M21,M31-M33.
```

## PHASE 4 | Fortinet, Cisco firewalls, Check Point adapters

Real config collection and deterministic parsing for 3 firewall families.

Tools/libraries this phase: HTTPX, Netmiko, vendor APIs, TTP/TextFSM

Suggested AI IDE skills/checklists: firewall-adapters, policy-normalization

Deliverables / exit gate: FortiOS, Cisco ASA/FTD and Check Point adapters, fixtures, capability manifests.

**COPY-PASTE PROMPT | PHASE 4**

```
Implement Fortinet FortiOS REST/SSH adapters, Cisco ASA CLI and FTD/FMC management API
adapters with exact family distinctions, Check Point management API/Gaia access where
applicable. Collect committed/effective policies, management services, ordered firewall
 rules, address/service objects, NAT and logs, firmware/model metadata. Add secure TLS
and SSH host-key validation; do not silently ignore certificates. Normalize
vendor-specific rule ordering/implicit action and distinguish absent API scope as
NOT_ASSESSED. Use fixture-backed positive/negative tests and optional authorized
virtual/physical lab verification. No device configuration writes in this phase. Cover
M05,M06,M07,M15 and start M20-M25.
```

## PHASE 5 | Sophos, Palo Alto and Cisco router/switch adapters

Complete remaining device families and switch hardening facts.

Tools/libraries this phase: PAN-OS, Sophos APIs, Netmiko, Scrapli, pyATS/Genie

Suggested AI IDE skills/checklists: router-switch-adapters, platform-capabilities

Deliverables / exit gate: Sophos/PAN-OS/IOS/IOS-XE/NX-OS adapters, parsing tests and vendor coverage report.

**COPY-PASTE PROMPT | PHASE 5**

```
Implement Sophos firewall XML/REST interface where supported by exact firmware, Palo
Alto PAN-OS REST/XML including committed policy and zones, Cisco IOS/IOS-XE routers and
 managed switches (NX-OS only where explicitly compatible). Use Netmiko/Scrapli,
TextFSM/TTP and Cisco pyATS/Genie where useful. Collect ACL, routing, SNMP, NTP,
Syslog, management access, VLANs, unused switch ports, port security/MAC binding, DHCP
snooping, Dynamic ARP Inspection and VLAN 1 management state. Implement complete
normalizers and secure/unsafe/unknown fixtures; capability matrix must list model and
software version support with proof. Cover M08-M11,M15,M26-M33.
```

## PHASE 6 | Full mandatory network-control evaluators and offline import

CIS control evidence for all explicit PS network requirements.

Tools/libraries this phase: Python ipaddress, ACL parser, deterministic check library

Suggested AI IDE skills/checklists: network-control-audit, offline-import

Deliverables / exit gate: Deterministic device-specific predicates; ACL shadowing; offline configs and rule examples.

**COPY-PASTE PROMPT | PHASE 6**

```
Implement device-specific normalized evaluators for M20-M33: Telnet/HTTP management
disabled where required, SSHv2/key-auth/timeouts, overly permissive ACLs, broad
sensitive-port exposure, ordered shadowed rules, default-deny, unused ports, switch
port security, DHCP snooping, ARP inspection, VLAN 1 management, Syslog, NTP,
SNMPv3/default strings. Implement effective ACL rule ordering and unknown semantics;
never use string search alone as proof of effective exposure. Create offline sanitized
config loader with OFFLINE provenance; negative tests include missing config,
vendor/firmware mismatch, parser ambiguity. Provide candidate findings with exact
expected/actual/evidence refs for Mandar policy engine. Cover M20-M33,E10.
```

## PHASE 7 | Remediation catalog, dry-runs, backup and recovery

Approved, allowlisted vendor/version-specific fixes with explicit capabilities.

Tools/libraries this phase: Ansible, Ansible Runner, OpenBao, vendor APIs

Suggested AI IDE skills/checklists: safe-remediation-catalog, backup-recovery

Deliverables / exit gate: Remediation candidate registry, dry-run/checkpoint capability tests and backups.

**COPY-PASTE PROMPT | PHASE 7**

```
Design vetted remediation templates keyed by vendor/model/firmware, control ID,
preconditions, expected state, risk, exact CLI/API commands, postcheck and
reversible/irreversible markers. Build no-write dry-run preview, expected diff, backup
artifact format, supported rollback and compensating/manual recovery runbook. Integrate
 Ansible Core, Ansible Runner, ansible.windows, Cisco/Fortinet/Palo Alto collections
only after version compatibility checks; PowerShell and vendor APIs as needed. Return
candidates to Mandar optimizer; never auto-generate commands from LLM or execute
browser-supplied raw text. Security tests reject unsupported firmware and
unapproved/malformed action. Cover M37,E21,E22,E24.
```

## PHASE 8 | Hybrid Batfish/custom simulation and safety contracts

Network and host what-if modeling for supported behaviors.

Tools/libraries this phase: Batfish, pybatfish, NetworkX, custom models

Suggested AI IDE skills/checklists: what-if-model-validation

Deliverables / exit gate: Batfish scenario runner, custom vendor semantics coverage and safety-contract tests.

**COPY-PASTE PROMPT | PHASE 8**

```
Implement Batfish + pybatfish snapshot creation and what-if comparison for supported
vendor/config versions; add custom deterministic policy models for missing
firewall/host behaviors (especially Sophos where unsupported by Batfish). Track ACL
order, NAT, objects, routes, source/destination, service flows, zone topology and model
 assumptions. Implement service-safety contracts with
SAFE_IN_MODEL/VIOLATES_CONTRACT/UNSUPPORTED/INSUFFICIENT_EVIDENCE results; blasts
radius as affected paths. Build differential fixtures and GNS3/EVE-NG tests wherever
licensed devices available. Do not label unsupported firmware universally safe. Return
simulation report to Mandar safety gate, no live write. Cover I11-I13.
```

## PHASE 9 | Controlled execution, staged rollout and recovery

Actual authorized remediation through safe orchestration.

Tools/libraries this phase: Ansible Runner, OpenBao, Pydantic actions, event logs

Suggested AI IDE skills/checklists: approved-hardening, staged-rollout

Deliverables / exit gate: Executor with approval checks, batch stops, rollback attempts and comprehensive journals.

**COPY-PASTE PROMPT | PHASE 9**

```
Implement device-engine bounded executor accepting only Mandar-issued signed/validated
approved ActionEnvelope IDs with asset/version/config preconditions and expiry.
Revalidate target identity, current state, credentials from OpenBao, backup readiness,
simulation verdict and approved action hash. Execute via Ansible
Runner/PowerShell/vendor API, log command template ID (redact secrets), enforce
idempotency, locks, timeouts and stop-on-failure. Stage rollout batch-by-batch and
block next batch on failed verification. Provide supported rollback/compensation and
NEEDS_MANUAL_RECOVERY for irreversible/unknown states. Default dry run; live writes
require explicit configured deployment mode and valid approval. Test against controlled
 lab fixtures/emulators, not production. Cover E21,E24,I15.
```

### PHASE 10 | Post-fix verification, drift analysis and evidence

Independent compliance recollect + service health tests + differential facts.

Tools/libraries this phase: SSH/TCP/HTTP probes, re-collection, diff and hashes

Suggested AI IDE skills/checklists: post-fix-verification, drift-evidence

Deliverables / exit gate: Verification JSON, drift events and evidence refs ready for proof generation.

**COPY-PASTE PROMPT | PHASE 10**

```
Implement independent post-change collection on same asset, audit evidence export,
selected CIS recheck requests and active service reachability checks (TCP/SSH/HTTP from
 authorized test points) with bounded timeouts. Return separate config_control_status
and service_contract_status; never report VERIFIED when one unknown/fails. Compute
normalized configuration diff and drift events with original/new hashes, timestamp,
evidence refs and source metadata. Coordinate with Mandar's proof-of-hardening and risk
 API; avoid DB writes from service/device-engine. Test known good/broken service
changes, unreachable device, misreported vendor, staged rollback. Cover E05,E25,I16.
```

### PHASE 11 | Vendor coverage tests, packaging and integration handoff

Run complete hardware/VM/fixture matrix and deliver connector package.

Tools/libraries this phase: pytest, Hypothesis, GNS3/EVE-NG, Linux/Windows VMs

Suggested AI IDE skills/checklists: vendor-matrix-QA, contract-handoff

Deliverables / exit gate: Installable package, CLI test harness, vendor matrix, proofs, clean API integration guide.

**COPY-PASTE PROMPT | PHASE 11**

```
Run pytest, hypothesis where appropriate, lint/types, static scans; contract tests
against packages/contracts/v1 fixtures. Document for every required
Windows/Linux/firewall/Cisco type: exactly tested version, collected facts, supported
controls, API/CLI permissions, simulation support, remediation/recovery support,
lab/fixture status and evidence link. Do not say fully tested on a vendor if only
fixtures ran. Provide sample JSON for
collect/evaluate/candidates/simulate/execute/verify, package installation guide,
OpenSCAP/ComplianceAsCode version pinning and licensing notes, CLI smoke tests, secrets
 redaction tests. Commit only services/device-engine and docs/device on
feat/vidhi-device. Return handoff and merge blockers.
```

## 6. Required feature ownership and dependencies

The table below includes all 88 IDs and single primary owner. A primary owner supplies tested functionality for that row; other teammates still implement their related API/UI/device integration. A feature is only complete end-to-end after all required layers pass integration tests.

## Mandatory PS - M01 to M38

| ID | Feature | Owner | Vidhi role |
|---|---|---|---|
| M01 | Windows 11 | Vidhi | PRIMARY |
| M02 | Windows Server 2019 | Vidhi | PRIMARY |
| M03 | Ubuntu 20.04/22.04 | Vidhi | PRIMARY |
| M04 | RHEL 8/9 | Vidhi | PRIMARY |
| M05 | Fortinet firewall | Vidhi | PRIMARY |
| M06 | Cisco firewall | Vidhi | PRIMARY |
| M07 | Check Point firewall | Vidhi | PRIMARY |
| M08 | Sophos firewall | Vidhi | PRIMARY |
| M09 | Palo Alto firewall | Vidhi | PRIMARY |
| M10 | Cisco router | Vidhi | PRIMARY |
| M11 | Cisco switch | Vidhi | PRIMARY |
| M12 | Live configuration collection | Vidhi | PRIMARY |

Mandatory PS - M01 to M38 (continued: 13-24)

| ID | Feature | Owner | Vidhi role |
|---|---|---|---|
| M13 | Central multi-target audits | Mandar | Interface / review |
| M14 | OS collectors | Vidhi | PRIMARY |
| M15 | Network collectors | Vidhi | PRIMARY |
| M16 | Deterministic CIS evaluation | Mandar | Interface / review |
| M17 | Modular YAML/JSON/XML rules | Mandar | Interface / review |
| M18 | Custom baselines | Mandar | Interface / review |
| M19 | Benchmark update maintenance | Mandar | Interface / review |
| M20 | Management plane security | Vidhi | PRIMARY |
| M21 | SSHv2/key authentication/timeout | Vidhi | PRIMARY |
| M22 | Over-permissive ACLs | Vidhi | PRIMARY |
| M23 | Sensitive-port exposure | Vidhi | PRIMARY |
| M24 | Shadowed rules | Vidhi | PRIMARY |

Mandatory PS - M01 to M38 (continued: 25-36)

| ID | Feature | Owner | Vidhi role |
|---|---|---|---|
| M25 | Default-deny policy | Vidhi | PRIMARY |

| ID | Feature | Owner | Vidhi role |
|---|---|---|---|
| M26 | Unused switch ports | Vidhi | PRIMARY |
| M27 | Port security / MAC binding | Vidhi | PRIMARY |
| M28 | DHCP snooping | Vidhi | PRIMARY |
| M29 | Dynamic ARP inspection | Vidhi | PRIMARY |
| M30 | VLAN 1 management traffic | Vidhi | PRIMARY |
| M31 | Syslog forwarding | Vidhi | PRIMARY |
| M32 | NTP synchronization | Vidhi | PRIMARY |
| M33 | SNMP security | Vidhi | PRIMARY |
| M34 | Central GUI dashboard | Apoorva | Interface / review |
| M35 | Compliance/severity breakdown | Apoorva | Interface / review |
| M36 | Expected vs actual finding | Apoorva | Interface / review |

Mandatory PS - M01 to M38 (continued: 37-38)

| ID | Feature | Owner | Vidhi role |
|---|---|---|---|
| M37 | Actionable remediation | Vidhi | PRIMARY |
| M38 | PDF/HTML/CSV/JSON reports and scale | Mandar | Interface / review |

## Industry extras - E01 to E30

| ID | Feature | Owner | Vidhi role |
|---|---|---|---|
| E01 | Asset inventory | Mandar | Interface / review |
| E02 | Authorized discovery | Mandar | Interface / review |
| E03 | Asset tagging/grouping | Mandar | Interface / review |
| E04 | Scheduled scans | Mandar | Interface / review |
| E05 | Configuration drift alerts | Vidhi | PRIMARY |
| E06 | Configuration snapshots | Mandar | Interface / review |
| E07 | Compliance trends | Mandar | Interface / review |
| E08 | Selective audits | Mandar | Interface / review |
| E09 | Distributed workers | Mandar | Interface / review |
| E10 | Offline configuration assessment | Vidhi | PRIMARY |
| E11 | CIS Level 1/2 profiles | Mandar | Interface / review |
| E12 | Manual review workflow | Mandar | Interface / review |

Industry extras - E01 to E30 (continued: 13-24)

| ID | Feature | Owner | Vidhi role |
|---|---|---|---|
| E13 | Risk-based prioritization | Mandar | Interface / review |
| E14 | Policy exceptions | Mandar | Interface / review |
| E15 | Role-based authorization | Mandar | Interface / review |
| E16 | Multi-factor authentication | Mandar | Interface / review |
| E17 | Device credential vault | Mandar | Interface / review |
| E18 | Read-only audit mode | Mandar | Interface / review |
| E19 | Application audit trail | Mandar | Interface / review |
| E20 | Policy version history | Mandar | Interface / review |
| E21 | Approved auto-remediation | Vidhi | PRIMARY |
| E22 | Dry-run preview | Vidhi | PRIMARY |
| E23 | Approval workflow | Mandar | Interface / review |
| E24 | Backup/recovery | Vidhi | PRIMARY |

Industry extras - E01 to E30 (continued: 25-30)

| ID | Feature | Owner | Vidhi role |
|---|---|---|---|
| E25 | Post-fix re-audit | Vidhi | PRIMARY |
| E26 | Alerts/webhooks | Mandar | Interface / review |
| E27 | Ticket integrations | Mandar | Interface / review |
| E28 | Executive reporting | Mandar | Interface / review |
| E29 | Search/filter interface | Apoorva | Interface / review |
| E30 | External integration API | Mandar | Interface / review |

### Innovations - I01 to I20

| ID | Feature | Owner | Vidhi role |
|---|---|---|---|
| I01 | Infrastructure relationship graph | Mandar | Interface / review |
| I02 | Cross-device correlation | Mandar | Interface / review |
| I03 | Configuration-based path analysis | Mandar | Interface / review |
| I04 | Root-cause reasoning | Mandar | Interface / review |
| I05 | Evidence-backed risk explanation | Mandar | Interface / review |
| I06 | Minimal-change optimization | Mandar | Interface / review |
| I07 | Alternative fix comparison | Mandar | Interface / review |
| I08 | Cross-vendor remediation planning | Mandar | Interface / review |
| I09 | Remediation conflict checks | Mandar | Interface / review |
| I10 | Change sequencing | Mandar | Interface / review |
| I11 | Cross-layer what-if simulation | Vidhi | PRIMARY |
| I12 | Service-safety contracts | Vidhi | PRIMARY |

Innovations - I01 to I20 (continued: 13-20)

| ID | Feature | Owner | Vidhi role |
|---|---|---|---|
| I13 | Blast-radius estimation | Vidhi | PRIMARY |
| I14 | Safety-gated execution | Mandar | Interface / review |
| I15 | Staged hardening rollout | Vidhi | PRIMARY |
| I16 | Dual verification | Vidhi | PRIMARY |
| I17 | Residual-risk analysis | Mandar | Interface / review |
| I18 | Proof-of-Hardening dossier | Mandar | Interface / review |
| I19 | Benchmark upgrade impact preview | Mandar | Interface / review |
| I20 | Explainable coverage decisions | Mandar | Interface / review |

# 7. Review checklist and merge plan

### Common integration order

• 1. Mandar exports contracts/v1 OpenAPI, Pydantic/JSON Schemas and sanitized fixtures; all three confirm compatible signatures.

• 2. Vidhi installs device-engine as an editable Python package inside Mandar backend worker environment; verify CLI and Pydantic contracts.

• 3. Mandar wires device jobs into Celery worker and checks evidence -> CIS evaluation -> findings/report using fixture and lab outputs.

• 4. Apoorva points typed API client to FastAPI and swaps mock mode to actual API; real Auth uses Supabase access tokens.

• 5. Run e2e path: login -> register -> audit -> finding -> graph -> plan -> simulation -> approval -> controlled change -> verification -> reports.

• 6. Test authorization and denied paths as thoroughly as success flows. No external device mutation in default automated CI.

• 7. Run full 88-ID coverage audit; any missing capability must be declared PARTIAL/BLOCKED, never marked implemented.

• 8. Produce device/version capability matrix and release notes. Validate PDFs/export counts against canonical audit database.

## Merge command pattern (after code review)

```
git switch main && git pull
# Merge in order: contracts/core, device engine, frontend
# Use protected PRs in GitHub; resolve schema/version conflicts explicitly.
python -m pytest         # backend/device when installed
pnpm -C apps/web test    # frontend tests
pnpm -C apps/web build   # frontend build
# Run docker compose integration and Playwright e2e in CI.
```

## Critical acceptance gates

• 38 mandatory entries are covered by documented platform/control tests, including Windows 11/Server 2019, Ubuntu 20.04/22.04, RHEL 8/9, five firewall vendors, Cisco routers/switches.

• No unassessed or unsupported control is considered PASS. Benchmark mapping and licensing/source provenance are visible.

• No privileged device command is sent without authorized scope, approved action, exact state preconditions and safety checks.

• Simulation states and model coverage are truthful; actual service checks are separate from predicted reachability.

• Four output formats (PDF/HTML/CSV/JSON) reconcile from the same canonical audit; reports contain expected/actual/evidence/remediation.

• 88 scope remains retained, but completion status must be evidence-backed. An irreversible change uses documented recovery, not fake rollback.

Handoff package - Vidhi: owned directory; pinned dependencies; install/run commands; no-secret .env.example; contract fixtures; test report; M/E/I evidence; known version limits; remaining integration questions.